Express.js — Day 11

Practical Middleware Patterns • Long-Term Reference Notes

Day 11 Goal

Understand how middleware can inspect requests, allow or reject them, run in a specific order, and attach information to **req** for later handlers.

1. Middleware: The Core Flow

Express middleware runs between the incoming request and the final route handler. A middleware function receives **req**, **res**, and **next**.

```
const middleware = (req, res, next) => {
// middleware work
next();
};
```

Pass: call **next()** to continue. **Fail:** send a response such as 401/403 and stop the request.

2. Authentication-Style Middleware

```
const checkAuth = (req, res, next) => {
const isLoggedIn = true;

if (isLoggedIn) {
next();
} else {
res.status(401).send("Please login");
}
};

app.get("/dashboard", checkAuth, (req, res) => {
res.send("Welcome to Dashboard");
});
```

When **isLoggedIn** is true, **next()** passes control to the route. When it is false, the middleware sends **401 Unauthorized** and the route handler does not execute.

3. Rejecting a Request with Middleware

```
const checkAge = (req, res, next) => {
const age = 15;

if (age >= 18) {
next();
} else {
res.status(403).send("Access denied");
}
};

app.get("/adult-content", checkAge, (req, res) => {
res.send("Adult content");
});
```

403 Forbidden is used here because the request is understood, but access is denied.

4. Route-Specific Middleware

```
app.get("/profile", checkAuth, (req, res) => {
res.send("Profile page");
});
```

Here, **checkAuth** is part of the middleware chain for **/profile**. It is not automatically attached to unrelated routes.

5. app.use() vs Route-Specific Middleware

Pattern	Meaning
`app.use(myMiddleware)`	Registers middleware broadly at that point in the application middleware chain. Requests that reach it can pass through it.
`app.get('/profile', myMiddleware, handler)`	Uses the middleware specifically in the middleware chain for the **/profile** GET route.

6. Middleware Execution Order

Express executes middleware and handlers in the order they are registered. This order matters.

```
const logger = (req, res, next) => {
console.log(req.method);
next();
};

const addUser = (req, res, next) => {
req.user = { name: "Bhavana" };
next();
};

app.get("/profile", logger, addUser, (req, res) => {
res.json(req.user);
});
```

Execution order: logger → addUser → route handler

7. Multiple Middleware on One Route

More than one middleware can be placed before the final route handler. Each middleware must call **next()** if it wants the request to continue.

```
app.get(
"/profile",
logger,
addUser,
(req, res) => {
res.json(req.user);
}
);
```

8. Middleware Can Modify req

Middleware can attach useful information to the request object. A later middleware or route handler can then read that information.

```
const addUser = (req, res, next) => {
req.user = {
name: "Bhavana",
role: "admin"
```

```
};

next();
};

app.get("/profile", addUser, (req, res) => {
res.json(req.user);
});
```

In this example, **req.user** is created/attached inside the middleware. The route can access it because the same request object continues through the middleware chain.

9. Object vs JSON Response

Inside Node.js, **req.user** is a JavaScript object. When **res.json(req.user)** is used, Express sends that data as a JSON response to the client.

```
req.user = {
name: "Bhavana",
role: "admin"
};

res.json(req.user);
```

Remember: JavaScript object → **res.json()** → JSON response.

10. Practical Day 11 Experiment — Role Check

We tested middleware using a query parameter so the request itself supplied the value being checked.

```
const checkRole = (req, res, next) => {
const role = req.query.role;

if (role === "admin") {
next();
} else {
res.status(403).send("Access denied");
}
};

app.get("/admin", checkRole, (req, res) => {
res.send("Welcome admin");
});
```

Test 1: `GET /admin?role=admin`

Observed result: **200 OK** — **Welcome admin**. Flow: request → checkRole → role is admin → **next()** → route handler → response.

Test 2: `GET /admin?role=user`

Observed result: **403 Forbidden** — **Access denied**. Flow: request → checkRole → role is not admin → send 403 → stop. The route handler does not execute.

11. Status Codes Used

Status	Meaning	Day 11 use
401	Unauthorized	Example authentication/login check.
403	Forbidden	Access denied, such as a failed role/age check.
200	OK	Successful request, such as an allowed admin request.

12. The Most Important Rule of Day 11

Every middleware has to decide what happens next:

```
Condition passes
↓
next()
↓
next middleware / route handler

Condition fails
↓
send response
↓
STOP
```

> If middleware neither calls **next()** nor sends/ends the response, the request can remain waiting.

13. Day 11 Assessment — Reference Answers

1. `app.use()` registers middleware broadly at that point in the application's middleware chain, while route-specific middleware is attached to a particular route.

2. With `age = 16`, the condition fails. The route handler does not execute, `next()` is not called, and the client receives **403 Forbidden** with **Access denied**.

3. Middleware can attach information to `req` so later middleware or route handlers can use that information during the same request.

4. The execution order is: **logger** → **addUser** → **route handler**.

14. Long-Term Mental Model

Think of middleware as checkpoints in a request pipeline. Each checkpoint can inspect the request, modify it, reject it, or allow it to continue.

```
Client
↓
Request
↓
Middleware 1
↓ next()
Middleware 2
↓ next()
Route Handler
↓
Response
```

> This same pattern becomes the foundation for real authentication, authorization, logging, validation, and other Express request-processing logic.

Day 11 Completion

Express.js Day 11 — COMPLETE ■

You practiced middleware in Postman, including allowing/rejecting requests and passing data through `req`.